

Levantamento complementar de melhorias

Projeto: C:\construtor-pg

Data de consolidacao: 2026-05-16

Documento complementar ao levantamento principal. Organiza melhorias por quatro frentes: tecnica, produto/governanca, performance/seguranca e riscos arquiteturais futuros.

Data: 2026-05-16

Projeto: `C:\construtor-pg`

Objetivo

Complementar o levantamento principal com uma segunda leitura, organizada em quatro frentes:

1. melhorias tecnicas;
2. melhorias de produto e governanca;
3. melhorias de performance e seguranca;
4. riscos arquiteturais futuros.

Este documento nao substitui o primeiro PDF. Ele amplia a visao para planejamento de medio prazo.

1. Melhorias tecnicas

1.1 Program Builder

- continuar a fatiar o `program-builder.js` em modulos menores;
- reduzir dependencias cruzadas entre editor, governanca, rebase, IA, import/export e API/Odoo;
- formalizar contratos internos para:
- estado do programa;
- estado da entidade;
- estado de governanca;
- diagnostico e preview.

Motivo

O builder cresceu muito. Hoje ele ja e um produto dentro do produto. Sem continuar a separacao, manutencao e regressao vao ficar caras.

1.2 Import/export

- separar mais claramente:
- definicao do mapping;
- preview;
- execucao;
- historico;
- agendamento;
- importacao XML;
- exportacao XML/TXT/CSV.

Motivo

Essa frente evoluiu rapido e tende a receber mais excecoes de negocio. Ela precisa continuar modular para nao virar um servico central inchado.

1.3 Canada administrativa

- reduzir dependencia de CRUD generico para operacoes criticas;
- criar superficies administrativas dedicadas para:
- grants;
- approvals;
- retencao;
- operacao de rebase;
- diagnostico de integridade.

Motivo

CRUD generico e bom para acelerar. Para governanca e manutencao critica, ele aumenta o risco operacional.

1.4 Contratos de metadata

- revisar e consolidar contratos de:
- `technicalProperties`;
- `navigation.query`;
- `publicationPolicy`;
- integridade estrutural;
- notificacoes administrativas;
- overlays.

Motivo

O projeto ganhou muitos contratos novos. Eles precisam permanecer coerentes entre backend, demo, runtime e builder.

1.5 Testabilidade

- isolar melhor pontos de orquestracao longa;
- facilitar testes de:
- publish governado;
- rebase;
- notificacao contextual;
- integridade apos CRUD administrativo.

Motivo

Hoje ja ha boa cobertura, mas alguns fluxos continuam exigindo testes maiores e mais caros.

2. Melhorias de produto e governanca

2.1 Politica clara de customizacao

- formalizar no produto quando usar:
- padrao puro;
- overlay;
- programa especifico/custom.

Motivo

Sem regra clara, o projeto pode cair no problema de ERP super customizavel: muito poder local, pouca governabilidade.

2.2 Politica de suporte

- definir o que entra em suporte oficial para:
- programa padrao;
- overlay;
- custom completo;
- integracoes do cliente.

Motivo

Essa decisao impacta diretamente custo operacional e expectativa do cliente.

2.3 Modelo de aprovacao

- definir melhor perfis e trilhas:
- quem pede;
- quem aprova;
- quem testa;
- quem publica;
- quem pode revogar ou congelar.

Motivo

Hoje a base tecnica existe, mas a governanca de processo precisa de definicao mais formal de papeis.

2.4 Estrategia de rollout

- definir quando uma mudanca vai:
- direto para todos;
- por tenant;
- por modulo;
- por feature flag;
- por fase de homologacao.

Motivo

Com ownership, overlays e governanca, o sistema ja esta perto de precisar rollout mais fino.

2.5 Politica de retencao e auditoria

- formalizar:
- quanto tempo guardar requests;
- quanto tempo guardar grants;
- quanto tempo guardar aprovacoes;
- quanto tempo guardar bundles;
- quanto tempo guardar checks e resigns de integridade;
- quando consolidar historico em vez de apagar.

Motivo

Isso afeta auditoria, LGPD, custo de armazenamento e suporte.

3. Melhorias de performance e seguranca

3.1 Performance do runtime metadata

- medir pontos de maior custo em:
- resolucao de telas;
- resolucao de overlays;
- abertura da Home;
- governanca no builder;
- listagens administrativas grandes.

Motivo

A complexidade funcional subiu. Precisa medir antes que lentidao vire problema estrutural.

3.2 Cache seletivo

- avaliar cache seguro para:
- resolucao de `screen\_definition`;
- `runtime\_endpoint`;
- algumas consultas do builder;
- metadata Odoo/API.

Motivo

Algumas leituras sao repetidas e relativamente estaveis. Pode haver ganho sem comprometer seguranca.

3.3 Hardening de seguranca

- revisar superficies de manutencao critica:
- reassinatura;
- publish governado;

- rebase;
- retencao;
- alteracao de parametros sensiveis.

Motivo

Quanto mais poder administrativo o sistema ganha, mais importante fica endurecer esses pontos.

3.4 Seguranca por ambiente

- ampliar politicas de:
- ambiente permitido;
- identidade do banco;
- bloqueio de operacao perigosa fora do ambiente esperado.

Motivo

Ja existe uma base boa disso. Vale aprofundar para reduzir erro humano.

3.5 Observabilidade

- melhorar monitoracao e logs para:
- falha de integridade;
- freeze/revoke de grant;
- publish bloqueado;
- rebase com conflito;
- execucao agendada de integracoes.

Motivo

Sem observabilidade boa, o sistema fica tecnicamente rico mas operacionalmente opaco.

4. Riscos arquiteturais futuros

4.1 Crescimento excessivo do builder

Risco

O construtor virar um monolito funcional com alto acoplamento entre UI, regras e operacao.

Mitigacao

- continuar fatiando modulos;
- formalizar contratos;
- manter smoke e testes por subdominio.

4.2 Multiplicacao de contratos dinamicos

Risco

Acumular muitos contratos JSON e metadata paralelos sem consolidacao, dificultando compatibilidade.

Mitigacao

- revisar contratos periodicamente;
- manter validadores e schemas;
- evitar variacoes equivalentes para o mesmo problema.

4.3 Excesso de modos de customizacao

Risco

Misturar padrao, overlay, custom, integracao e execucoes locais ate perder previsibilidade.

Mitigacao

- politica clara de quando usar cada modo;
- limites objetivos;
- rejeitar customizacao fora do contrato sempre que possivel.

4.4 Dependencia operacional de poucos fluxos administrativos

Risco

Se governanca, integridade e publish dependerem demais de conhecimento implícito, o sistema fica difícil de operar por outros perfis.

Mitigacao

- telas dedicadas;
- timeline;
- notificacoes com acao direta;
- documentacao operacional mais objetiva.

4.5 Complexidade crescente de import/export

Risco

Import/export virar um subproduto grande demais, com regras demais misturadas na mesma engine.

Mitigacao

- modularizacao continua;
- escopo claro por formato;
- evitar recurso “universal” sem contrato fechado.

4.6 Acoplamento entre demo, mock e producao

Risco

Mesmo com a matriz de paridade, as frentes podem continuar crescendo e espalhando superfície de demo.

Mitigacao

- seguir usando a matriz de paridade;
- manter smoke local separado da entrada real;
- revisar periodicamente o que ainda precisa existir como demo.

Recomendacao de uso no plano

Curto prazo

- fechar Git da frente atual;
- reforçar testes integrados;
- consolidar governanca e integridade.

Medio prazo

- continuar fatiando builder e import/export;
- fortalecer superficies administrativas dedicadas;
- ampliar observabilidade e politicas de ambiente.

Longo prazo

- controlar a expansao de customizacao;
- evitar explosao de contratos dinamicos;
- manter o produto governavel, mesmo com alto poder de configuracao.

Conclusao

O projeto ja passou da fase em que o maior risco era “faltar funcionalidade”. Agora os riscos principais sao:

- complexidade acumulada;
- operacao administrativa;
- governabilidade;
- manutencao sustentavel;
- previsibilidade de evolucao.

Por isso, a tendencia correta daqui para frente e equilibrar novas features com: